

Network Boundaries

In-Process Conceptual Boundaries: just call Java code

Network (and Process) Boundaries: not so simple

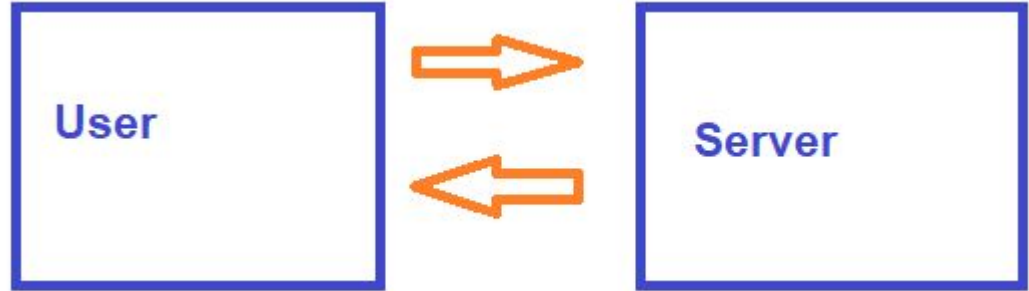
- Data must be serialized
- Error codes must also be serialized

Communication

How do the orange arrows actually work?

Server: hostname (ip address)
+ port

User: sends data to that port,
using the correct **protocol**



Multiple Java Processes

Same process = shared memory, use memory data (objects, etc) to communicate

Different processes = no shared memory

⇒ Most basic cross-process communication: File system

File System communication requires that the processes share a file system, this won't work for multi-server setup

File System Communication

Two processes = (at least) two threads

Locks are objects

⇒ What to use instead?

Atomic File System operations

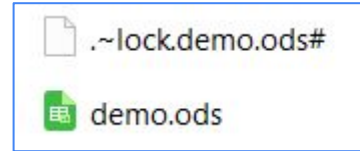
Note: There's a reason this is unusual in practice

File System Communication - Not on Exam

Most common “lock”: file existence

- May have seen with older programs

Also useful: `rename()` is atomic



Typical pattern:

- Write contents to a temporary file
- Rename the fully written file to some agreed upon cross-process name
- Other process checks for existence of file, reads contents if it exists

Your Turn!

1. Clone the MultiProcessExercises repo from the link on Brightspace
2. Use the code from the exercise1 package
3. Fill out the 'writeFileContents' method in DataSender.java
4. Fill out the 'readFileContents' method in DataReceiver.java
5. Run both programs at once and see what happens
6. Put a breakpoint in DataSender in writeFileContents and DataReceiver in readFileContents, and walk through line by line; what actually triggers the breakpoint in DataReceiver?

Serialization Protocols

Many, many options. Some common ones:

XML - very verbose, allows for easy automated schema validation. Largely replaced by more compact formats

Example: Eclipse .classpath file

JSON - compact, human-readable

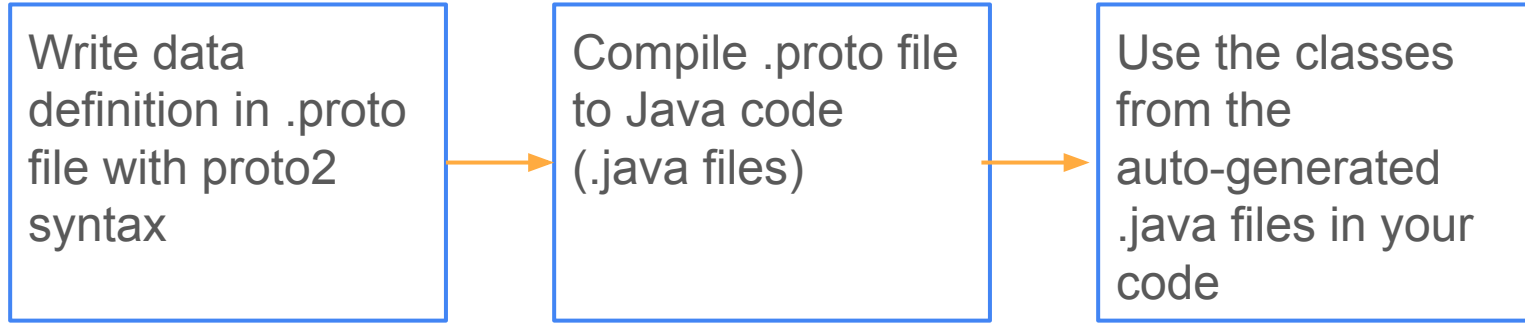
YAML - semantic whitespace, otherwise very similar to JSON

Example: Github Actions config files

Protocol buffers - multiple modes (very compact machine readable bits, compact human-readable format)

This is what we'll be using

Writing Java Code With Protocol Buffers



Protocol Buffer Syntax

Full spec: <https://protobuf.dev/reference/protobuf/proto2-spec/>

Version note: we'll be using proto2 (still the default); proto3 has some rough edges

Most common aspects:

int $\Rightarrow$ int32

long $\Rightarrow$ int64

String $\Rightarrow$ string

boolean $\Rightarrow$ bool

Protocol Buffer Syntax

class $\Rightarrow$ message

enum $\Rightarrow$ enum

List, [] array $\Rightarrow$ repeated

Map $\Rightarrow$ map

Protocol Buffer Syntax Example

```
public class SampleMessage {  
  
    private int id;  
    private List<String> values;  
    private ResultCode resultCode;  
  
    public enum ResultCode {  
        SUCCESS,  
        FAILURE  
    }  
}
```

```
message SampleMessage {  
    optional int32 id = 1;  
    repeated string values = 2;  
    optional ResultCode resultCode = 3;  
}  
  
enum ResultCode {  
    UNKNOWN = 0;  
    SUCCESS = 1;  
    FAILURE = 2;  
}
```

Backwards-compatibility:

- Declare all fields as optional or repeated, never required
- Always have an 'unknown' option for enums

Compiling .proto -> .java

Equivalent of javac: protoc

- Does not have the best usability
- We already have a build configuration management tool that calls javac

⇒ Better solution: have gradle call protoc for you

Compiling .proto -> .java

Configure protobufs in build.gradle

add the plugin (this also configures grpc for later):

```
plugins {  
    id 'com.google.protobuf' version '0.9.4'  
}  
def grpcVersion = '1.59.1'  
def protobufVersion = '3.25.1'  
def protocVersion = protobufVersion  
protobuf {  
    protoc { artifact = "com.google.protobuf:protoc:${protocVersion}" }  
    plugins {  
        grpc { artifact = "io.grpc:protoc-gen-grpc-java:${grpcVersion}" }  
    }  
    generateProtoTasks {  
        all().*.plugins { grpc {} }  
    }  
}
```

Compiling .proto -> .java

Configure protobufs in build.gradle:

add the source sets - this uses a folder next to 'src' and 'test' called 'protobuf':

```
main {  
    proto {  
        srcDirs = ['protobuf']  
    }  
    java {  
        srcDirs = ['src',  
                  'build/generated/source/proto/main/grpc',  
                  'build/generated/source/proto/main/java']  
    }  
}
```

Caution: Don't remove the test source sets!

Compiling .proto -> .java

Configure protobufs in build.gradle:

add the dependencies:

```
implementation "io.grpc:grpc-protobuf:${grpcVersion}"  
implementation "io.grpc:grpc-services:${grpcVersion}"  
implementation "io.grpc:grpc-stub:${grpcVersion}"  
compileOnly "org.apache.tomcat:annotations-api:6.0.53"
```

```
runtimeOnly "io.grpc:grpc-netty-shaded:${grpcVersion}"
```

```
testImplementation "io.grpc:grpc-testing:${grpcVersion}"  
testImplementation "io.grpc:grpc-inprocess:${grpcVersion}"
```

Compiling .proto -> .java

Refresh Eclipse metadata:

```
./gradlew eclipse
```

To recompile the protos (after any change to a .proto file), run:

```
./gradlew generateProto
```

and refresh Eclipse

Auto-Generated Code and Version Control

Best Practice: Don't commit compiled files, let the build configuration handle that

Ex: don't commit .class files, they'll just cause merge conflicts and Eclipse errors

For protos, don't commit the .java files, just the .proto files

⇒ Use your .gitignore to avoid committing the 'generated' directories

Auto-Generated Code and Version Control

How to get the .java files back so that github can compile the code?

Update the gradle.yml!

Add

```
./gradlew generateProto;
```

Before the

```
./gradlew build
```

How to Use Compiled Protocol Buffers

Protoc generated classes:

- create
- read
- write

Note the **builder** pattern!

```
public static void main(String[] args) throws Exception {
    Sample.SampleMessage message = Sample.SampleMessage.newBuilder()
        .setId(1)
        .addValues("Hello World")
        .addValues("Another String")
        .setResultCode(Sample.ResultCode.SUCCESS)
        .build();

    System.out.println(message);

    ByteArrayOutputStream baos = new ByteArrayOutputStream();
    message.writeTo(baos);
    baos.close();

    byte[] data = baos.toByteArray();

    Sample.SampleMessage roundTrip = Sample.SampleMessage.parseFrom(data);

    System.out.println(roundTrip);
}
```

How to Use Compiled Protocol Buffers

Optional, often useful: Add a translation layer

The classes protoc generates are very data-layer-only; you can use them as-is, but often it's helpful to create an internal data representation

- This is especially true for long-lived protos that have backwards compatibility cruft

```
public static SampleMessage parseFromProto(Sample.SampleMessage message) {  
    int id = message.hasId() ? message.getId() : 0;  
    List<String> values = new ArrayList<>(message.getValuesList());  
    ResultCode code = message.hasResultCode() ?  
        ResultCode.parseFromProto(message.getResultCode()) : ResultCode.SUCCESS;  
    return new SampleMessage(id, values, code);  
}
```

Your Turn!

1. Use exercise2 from the MultiProcessExercises repo
2. Create a .proto file that defines a Request and a Response message for the server (check out the TODOs for details)
3. Compile the .proto with `./gradlew generateProto`
 - a. You'll need to declare a package for the output; use `package <package name>;` at the top of the .proto just as you would for a java file
4. Use the proto messages from (3) to fix the TODOs in Server
5. Use the proto messages from (3) to fix the TODOs in TestServer
6. Verify that the test passes

Writing Multiple Protos

Common Use Case:

For a single message, use **writeTo**

- This includes messages wrapped in an outer message, ex:

```
message ProtoList {  
    repeated InnerProto = 1;  
}
```

Unusual:

To write many messages, use **writeDelimitedTo**

Ex: Mix-and-match serialization of different types

Your Turn!

1. Re-use the code from exercise1
2. Change the FileWriter -> FileOutputStream and FileReader -> FileInputStream (Writer/Reader = characters, Input/OutputStream = bytes)
3. Fill out the 'writeFileContents' method in DataSender.java **to write a protocol buffer message (can use any already-created ones)**
4. Fill out the 'readFileContents' method in DataReceiver.java **to read a protocol buffer message**
5. Run both programs at once and see what happens

Protos as Persistence Format

Storing data in a database or other long-lived storage layer

Backwards Compatibility gets even more exciting:

- What version was written?
- Unless all data can be (and is) upgraded, arbitrarily long support window

Network compatibility is usually for just one version, this is harder

Advantages: compact, can be easily parsed into human-readable form

For long-term persistence, **always include a version number**

Backwards Compatibility

Old data, new code:

Ex: Old data is loaded out of a database by a new server

Ex: Un-upgraded client sends a request to an upgraded server

- Missing newer fields $\Rightarrow$ all new fields must have a sensible **default** behavior
- Data in older fields $\Rightarrow$ all **deprecated** fields must have an on-the-fly migration path (either ignored or upgraded)

Backwards Compatibility

New data, old code:

Ex: Upgraded server sends a response to an un-upgraded client

- Missing older fields $\Rightarrow$ all older (supported) fields must still be populated
- Data in newer fields $\Rightarrow$ all new data must be safe to **ignore** for older clients

Backwards Compatibility Best Practices

Never re-use a **field number**

- Easy solution: when adding a new field, increment by one from the last field
- This includes changing types/names; any changes are modeled as a 'delete and add' and count as re-using the field number

Always include a **version** number

- This should be hardcoded in a .java file, ex: `setVersion(VERSION)`
- The version should increment any time the persistence format changes

Your Turn!

1. Create a `TextEditorPreferences` message in a `.proto` file, with a string for preferred font and a bool for light vs dark mode.
2. Generate the `.java` files for this proto
3. Create a `PreferenceStorer` java class with a main method:
 - a. Create a `TextEditorPreferences` object
 - b. Store the object to a file as a proto message
 - c. Read the file back in, and populate a new object - print old vs new to the console to verify they're the same
4. Update the `TextEditorPreferences` message in the `.proto` file - add an `int32` for preferred font size
5. Regenerate the `.java` files for this proto
6. Update `PreferenceStorer` to not write to the file (so you'll be reading the old, pre-step-4 format message)
7. Verify that you can still read the old format back into an object

Multiple Java Processes

Cross network is similar to cross process but instead of 'writing to a file', code is 'writing to a port'

Same set of problems as files:

- Notice when there is data
- Get all the data, not just part of it
- Parse the data into something useful

Network Communications

To send data to a server on a port, the client requests a connection and the server opens a **socket**

- this alerts the server that data is incoming
- allows data to pass back and forth from client $\Leftrightarrow$ server

The socket is the rough equivalent of the shared file used previously

Multiple Java Processes

The low-level hardware side is a commonly solved problem, lots of libraries

- We'll be using gRpc (google remote procedure call)

gRpc, server-side:

- Handles socket requests and creation (including authentication)
- Reads all the data
- Parses the bytes into a protocol buffer message
- **Sends the message to your code**

Multiple Java Processes

The low-level hardware side is a commonly solved problem, lots of libraries

- We'll be using gRpc (google remote procedure call)

gRpc, client-side:

- Handles socket requests and creation (including authentication)
- Turns a protocol buffer message into bytes
- Sends the bytes to server
- Gets a response back as bytes
- Parses those bytes into a protocol buffer message
- **Returns the message to your code**

Endpoint Definitions

Allows one port to respond to many different requests

Typically implemented via a REST API

- Common (not universal) practice: server calls are **stateless**, meaning the client must manage all state between calls
- allows for easy handling of multiple clients at once and graceful client disconnects

Different methods on the server correspond to different paths (often part of a URL)

ex: `https://graph.facebook.com/facebook/picture?redirect=false`

gRPC & Protobuf: Peanut Butter & Jelly

Interoperable configuration language to specify

- Endpoints (gRpc)
- Serialization format (protobuf)

Compiles to many languages, including (but not limited to):

- Java
- C++
- Python
- Typescript (which compiles to Javascript)

Using gRPC

Pieces of the server:

Endpoint definitions - .proto file

Message definitions - .proto file

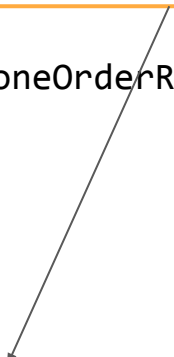
Server definition - .java file

Client definition - .java file (or other language!)

Endpoint Definition - Just an API!

In the same way that classes translate to proto messages, APIs (interfaces) translate to a gRPC service.

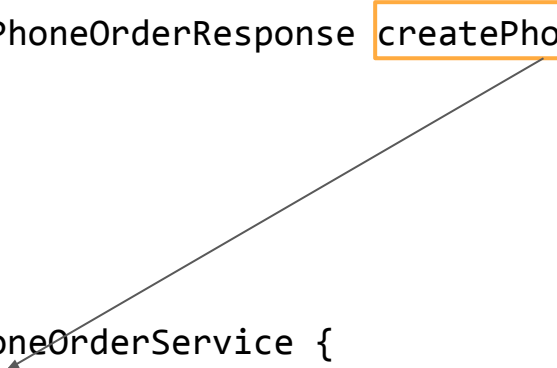
```
public interface PhoneOrderService {  
    public PhoneOrderResponse createPhoneOrder(PhoneOrderRequest request);  
}  
  
service PhoneOrderService {  
    rpc CreatePhoneOrder (PhoneOrderRequest) returns (PhoneOrderResponse) {}  
}
```

A diagram consisting of a thin black arrow pointing from the 'PhoneOrderService' interface definition in the first code block to the 'PhoneOrderService' service definition in the second code block, illustrating the translation of the API into a gRPC service.

Endpoint Definition - Just an API!

In the same way that classes translate to proto messages, APIs (interfaces) translate to a gRPC service.

```
public interface PhoneOrderService {  
  
    public PhoneOrderResponse createPhoneOrder(PhoneOrderRequest request);  
  
}  
  
service PhoneOrderService {  
    rpc CreatePhoneOrder (PhoneOrderRequest) returns (PhoneOrderResponse) {}  
}
```

A diagram consisting of a thin black line with an arrowhead at the bottom. It originates from the 'createPhoneOrder' method name in the 'PhoneOrderService' interface and points to the 'CreatePhoneOrder' RPC name in the 'PhoneOrderService' gRPC service definition, illustrating the mapping between the two.

Endpoint Definition - Just an API!

In the same way that classes translate to proto messages, APIs (interfaces) translate to a gRPC service.

```
public interface PhoneOrderService {
```

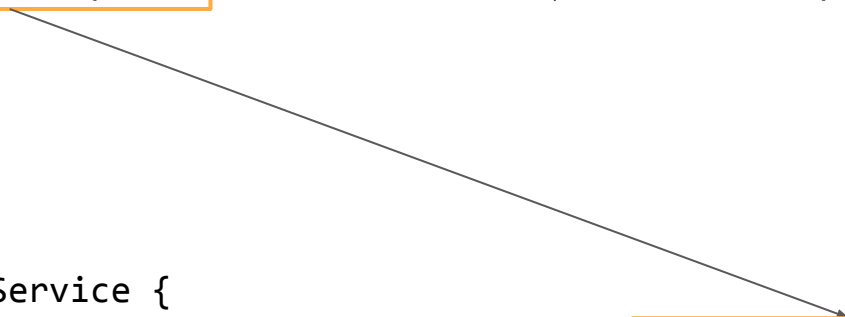
```
    public PhoneOrderResponse createPhoneOrder(PhoneOrderRequest request);
```

```
}
```

```
service PhoneOrderService {
```

```
    rpc CreatePhoneOrder (PhoneOrderRequest) returns (PhoneOrderResponse) {}
```

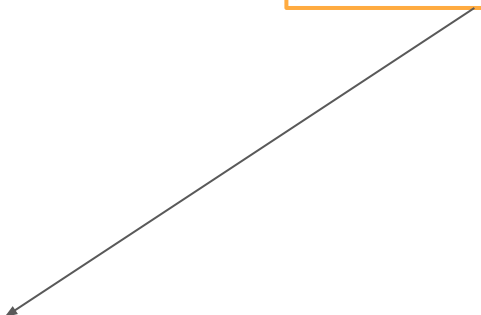
```
}
```



Endpoint Definition - Just an API!

In the same way that classes translate to proto messages, APIs (interfaces) translate to a gRPC service.

```
public interface PhoneOrderService {  
  
    public PhoneOrderResponse createPhoneOrder(PhoneOrderRequest request);  
  
}  
  
service PhoneOrderService {  
    rpc CreatePhoneOrder (PhoneOrderRequest) returns (PhoneOrderResponse) {}  
}
```



A diagram consisting of a thin black line with an arrowhead at the bottom. It originates from the `PhoneOrderRequest` parameter in the `createPhoneOrder` method of the `PhoneOrderService` interface and points to the `(PhoneOrderRequest)` parameter in the `CreatePhoneOrder` RPC of the `PhoneOrderService` service definition.

What Does This Create?

To the IDE!

- Run `./gradlew generateProto`
- Manually refresh the project in the IDE

Client/Server Definition - Boilerplate

Server/Client code can be treated as a template - included on Brightspace

"Interesting" code lives in the actual client method call (order() in the example),
and the <serviceName>ServiceImpl on the server

Server Definition

Create the actual implementation for the API methods:

- Create a class that extends the `<serviceName>ImplBase`
- Override each rpc method with an actual implementation
- Use `onNext/onCompleted` to return a value

Caution: Don't use `onError` - that's for something going wrong at the networking layer

Server Definition

Start up the server on a particular port:

- For simple cases, no login is required, so don't require credentials
- Pick a unique-ish port (for an actual production server, this would be specified in a config file, not the actual binary)

Client Definition

gRPC does not require a specific client language - you can use any client you want!

Some useful examples:

Command line - grpc and curl often don't play well, use grpcurl instead
(<https://github.com/fullstorydev/grpcurl>)

Ex: `grpcurl -plaintext -d '{"model": "android"}' localhost:50051 phoneorder.PhoneOrderService/CreatePhoneOrder`

Returns

```
{  
  "orderNumber": "not implemented yet"  
}
```

Client Definition

Java Client: Similar to server, use a template

Once you have a <servicename>Client variable:

- call methods on it as defined in the .proto file
- Usually, create request messages, check response messages
- Can prompt the user for input with a Scanner/report information with println

Your Turn!

Create an AddingMachine server, and query it with either a java client or curl:

1. Create a .proto file with
 - a. an AddingRequest that includes two integers
 - b. an AddingResponse that includes an integer result and an error code (remember the optional keyword!)
 - c. an AddingMachine service with an add rpc that takes an AddingRequest and returns an AddingResponse
2. Generate all the proto and gRPC files
3. Create an AddingMachineServer.java that starts up an AddingMachine service
4. Create a client, or use grpcurl, to verify that your server can correctly add $2 + 3$

Let's Review, Serialization & Protobuf

- 3 major serialization formats
- How to specify int, long, String, and boolean in a .proto file
- How to handle backwards compatibility with protocol buffer: adding new fields, removing old fields, including a version number
- How to specify a message in a .proto file

Let's Review, gRPC

- How to specify a service in a .proto file
- What happens when sending a message from client to server with gRPC (open a socket, serialize/deserialize data, connect to your code)
- How to return a value from the .java implementation of a gRPC service
- An advantage of multi-process computing over multi-threaded-but-single-process computing
- Ways to design for multi-process computing